TOC Case Study

Team:13

Members:

- CB.SC.U4CSE24107
- CB.SC.U4CSE24115
- CB.SC.U4CSE24148
- CB.SC.U4CSE24160

Question No: 1

Prompt used:

Role

Act as a Theory of Computation (TOC) subject expert who can solve scenario-based automata problems accurately and explain state-transition diagrams clearly.

Task

I will provide a TOC case study question with one or more divisions. Analyse each division carefully and construct an automaton that satisfies the given conditions. Keep the answer concise, clear, and suitable for a student's academic submission.

Requirements

For each division, follow these steps:

1. **Interpret the question:** Identify the input alphabet, conditions, restrictions, and acceptance criteria.
2. **Define the language:** Express the language mathematically if required and explain which strings are accepted.
3. **Choose the automaton:** Select the appropriate model (DFA, NFA, ε-NFA, PDA, or TM) based on the problem. Briefly justify the choice.

4. **Formal definition:** Provide the required mathematical definition, including states, input alphabet, transition function, initial state, and accepting states. Include stack details for a PDA where necessary.
5. **Explain the states:** Briefly describe the purpose of each state, including any trap state.
6. **Transition table:** Provide a complete and correct transition table. For a DFA, include a transition for every state and input symbol.
7. **State-transition diagram:** Provide a clear diagram using Mermaid or Graphviz DOT syntax. Include the initial state, accepting states, labelled transitions, self-loops, and trap state wherever required. If possible, provide a rendered diagram suitable for my report.
8. **Working and examples:** Explain how the automaton processes the input. Trace at least two accepted and two rejected strings, where applicable.
9. **Verification:** Check that the formal definition, transition table, and diagram agree and that the automaton correctly handles invalid inputs and boundary cases.

Important Instructions

- Derive the automaton from the actual scenario. Do not assume a DFA is always appropriate.
- Avoid unnecessary states, but do not omit states needed for correctness.
- Do not invent conditions or transitions. Clearly mention any assumptions required.
- Keep the explanation concise and student-like, using simple academic language.
- Preserve the original question numbering and solve each division separately.
- Ensure the solution is technically accurate and easy to understand, reproduce, and include in my case study report.

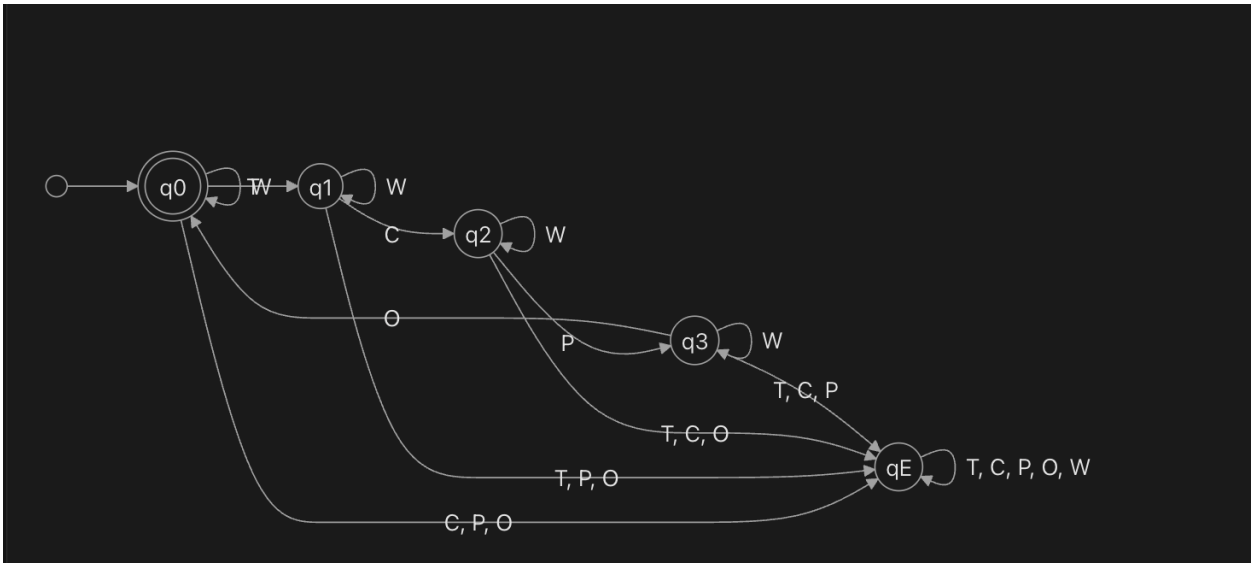
Expected Output

Present each solution in this order: interpretation, language definition (if required), automaton choice, formal definition, state explanations, transition table, diagram with source code, example traces, and correctness verification.

State Diagram:

State	T	C	P	O	W
→ *q_0	q_1	qE	qE	qE	q_0
q_1	qE	q_2	qE	qE	q_1
q_2	qE	qE	q_3	qE	q_2
q_3	qE	qE	qE	q_0	q_3
qE	qE	qE	qE	qE	qE

Automaton:



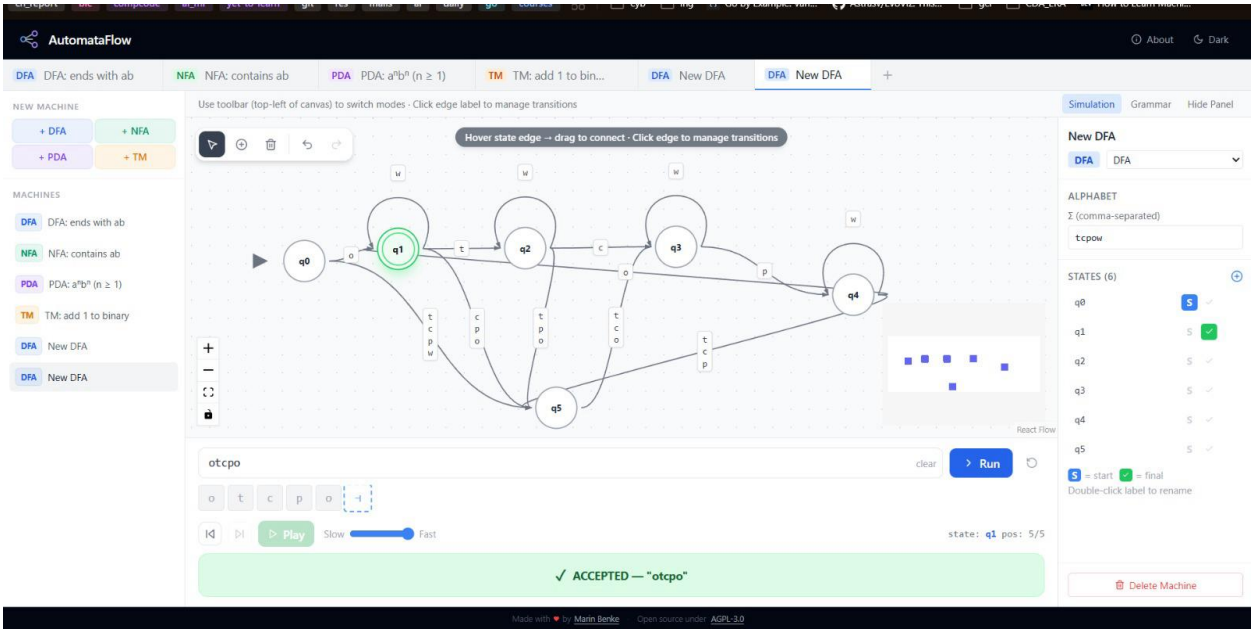
Comparison:

Both the AI's automaton and the team's automaton enforce the same rules: T, C, P and O must occur in that order, W loops in every stage, any ordering violation leads to a permanent trap state, and a sequence is accepted only when a full cycle has been completed. Once the first o is consumed, states q1 to q5 of the team's automaton behave exactly like q0 to qE of the AI's automaton, and all aligned test strings gave matching results. The team's language is therefore L(team) = O · L(AI). The difference

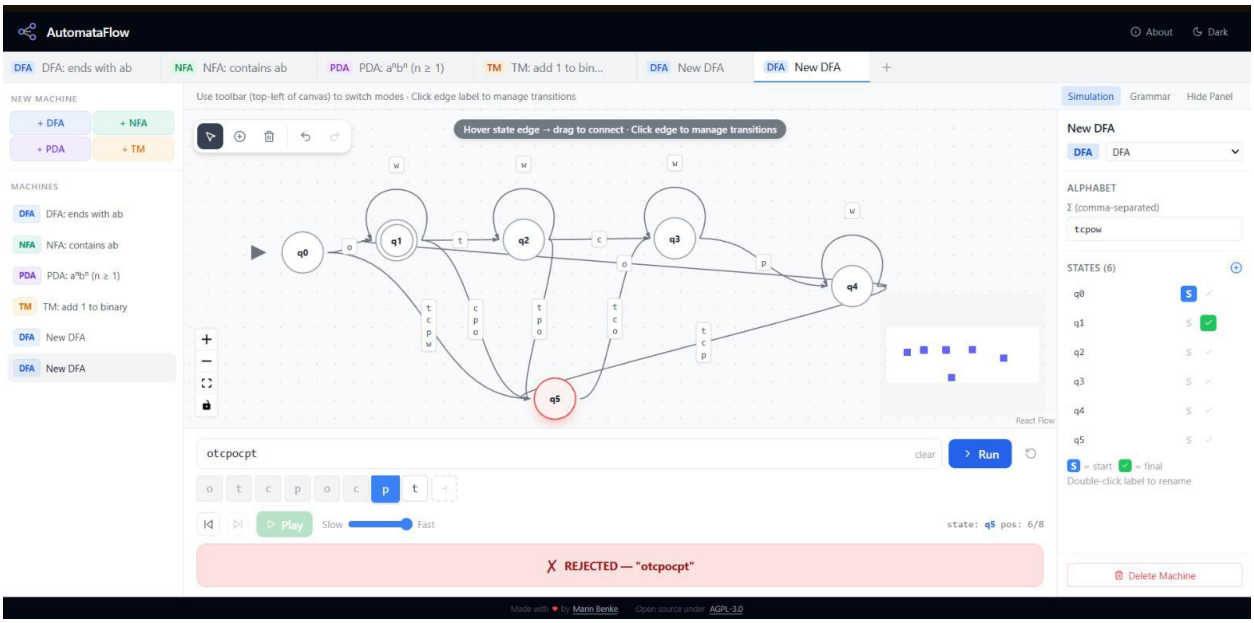
lies only at the start. In the AI's automaton, the initial state is the safe state and is also accepting. This follows the wording of the question ("initially, the gate is open" and "ends in its initial safe state"), so the empty string and strings beginning with T or W are valid, while a leading O is an error. The team's automaton instead treats the start as a separate state, q0, which is non-accepting and requires a "gate confirmed open" event (o) before anything else. As a result, it rejects the empty string, rejects strings beginning with t or w, and accepts only strings beginning with o. Because of this, the two languages have no accepted strings in common, even though their cycle logic is identical. The accepting state of the team's automaton (q1) is also not its initial state, which departs from the question's requirement that a valid sequence ends in the initial safe state. The team's design is still reasonable if the controller must first confirm that the gate is open, but this would need to be stated as an explicit assumption in the report.

AUTOMATON SIMULATOR OUTPUT:

1)Valid String



2)Invalid string:


AutomataFlow
About
Dark
DFA DFA: ends with ab
NFA NFA: contains ab
PDA PDA: a^n b^n (n ≥ 1)
TM TM: add 1 to bin...
DFA New DFA
DFA New DFA
NEW MACHINE
+ DFA
+ NFA
+ PDA
+ TM
MACHINES
DFA DFA: ends with ab
NFA NFA: contains ab
PDA PDA: a^n b^n (n ≥ 1)
TM TM: add 1 to binary
DFA New DFA
DFA New DFA
Use toolbar (top-left of canvas) to switch modes · Click edge label to manage transitions
Hover state edge → drag to connect · Click edge to manage transitions
q0
q1
q2
q3
q4
q5
React Flow
otcpocpt
clear
Run
o t c p o c p t
Play
Slow
Fast
state: q5 pos: 6/8
REJECTED — "otcpocpt"
Simulation
Grammar
Hide Panel
New DFA
DFA
DFA
ALPHABET
Σ (comma-separated)
tcpow
STATES (6)
q0
q1
q2
q3
q4
q5
S = start ✓ = final
Double-click label to rename
Delete Machine
Made with ♥ by Marin Benke
Open source under AGPL-3.0